based on scopes, granting the third party (the OAuth2 Client) specific permissions depending on what scopes have been assigned.

Regarding Edge Service, we can decide upfront what scopes it should be granted. This section will show you how to configure a `roles` claim with the list of roles assigned to the authenticated user. Then you'll use a `roles` scope to grant Edge Service access to that claim and instruct Keycloak to include it in both ID Tokens and Access Tokens.

Before moving on, you'll need a Keycloak container up and running. Open a Terminal window, navigate to the folder where you keep your Docker Compose file, and run the following command:

```
$ docker-compose up -d polar-keycloak
```

If you haven't followed along, you can refer to Chapter12/12-begin/polar-deployment/docker/docker-compose.yml in the accompanying repository.

NOTE Later I'll provide you with a JSON file you can use to load the entire configuration when starting up the Keycloak container without being concerned with its persistence (as I did in the previous chapter). If you want to follow this second option directly, I still invite you to read through this section, since it provides the essential information you'll need when moving on to the Spring Security part of the chapter.

CONFIGURING ACCESS TO USER ROLES IN KEYCLOAK

Keycloak comes preconfigured with a `roles` scope you can use to give an application access to the user roles contained in a `roles` claim. However, the default representation of the roles list is not very convenient to use because it's defined as a nested object. Let's change that.